Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval)

A Practical Evaluation Procedure for Neural Network Optimization Algorithms

Silver Moon (`cloverxion`)
Lumina Moon (`cloverx-id`)

November 2026
Technical Specification — Version 1.1.1

Abstract

This specification defines the **Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval v1.1.1)**, a practical procedure for testing and benchmarking neural network optimizers across CPU and GPU implementations. LSO-Eval divides verification into four sequential stages, moving from fast mathematical checks on CPU to full-scale model training on GPU: (I) Algebraic Safety and Numerical Precision Checks, (II) Synthetic Test Functions, Backend Consistency, and Checkpointing, (III) Short-Run Proxy Model Screening, and (IV) Large-Scale Pre-Training Evaluation with Train–Validation Deduplication and Two-Class Acceptance Criteria. Beyond the core language modeling pipeline, **V-Eval** tests computer vision architectures across low-, medium-, and full-resolution datasets, and **D-Eval** guards against regressions between software releases. Both Stage IV and V-Eval define a single-GPU *Standard Setup* alongside a multi-GPU *Extended Setup* so the same benchmark rules scale from local workstations to cluster runs. All rules in Sections 1–7 are algorithm-agnostic.

---

1 Scope, Principles, and Evaluation Stages

1.1 Objectives and Scope

LSO-Eval provides a shared testing checklist for optimization algorithms and CPU/GPU compute kernels. Because the procedure does not depend on any single update rule, its specification version (v1.1.1) is tracked separately from individual optimizer releases. The methodology focuses on four practical goals:

1. **Save Compute Resources:** Catch division-by-zero risks, numerical overflow or underflow, and narrow hyperparameter ranges on CPU before spending GPU hours.
2. **Separate Mathematical Issues from Kernel Bugs:** Check symbolic update equations, function approximations, low-precision rounding, and parallel compute kernels in separate steps so the cause of any failure is easy to locate.
3. **Verify Backend and Multi-GPU Consistency:** Confirm that multithreaded CPU kernels, SIMD vector paths, GPU compute shaders or kernels, sharded multi-GPU reductions, and Python bindings match a single-threaded scalar reference within defined numerical tolerances.

4. **Jointly Evaluate Model Quality and Optimizer Memory:** Pair validation metrics with isolated optimizer state memory—measured separately from model weights, gradients, and activations—using distinct acceptance criteria for two-buffer (full-state) and single-buffer (compressed-state) designs.

1.2 Evaluation Stages

LSO-Eval is organized into a fast CPU verification phase (**Stages I and II**) and a GPU training phase (**Stages III, IV, and V-Eval**), supplemented by version regression tests (**D-Eval**) as summarized in Table 1. The CPU stages act as an automated gate that catches mathematical or kernel-parity defects in under a minute before any GPU budget is spent. The GPU stages then evaluate training stability, memory footprint, and throughput from single-GPU setups up to multi-node clusters. Pipeline execution is strictly sequential: if a candidate fails any stage, evaluation halts immediately until the underlying issue is resolved.

Table 1: The LSO-Eval Evaluation Pipeline (v1.1.1)

Stage	Focus	Main Checks	Typical Runtime
Stage I	Math & Precision	No division by zero, bounded update magnitude, half-precision floor safety, function approximation accuracy, and unbiased low-precision rounding.	Seconds (CPU)
Stage II	Synthetic & Parity	Convergence on ill-conditioned functions ($\kappa \geq 1000$), saddle escape, hyperparameter stability, single-pass reduction consistency, cross-backend/sharded parity, and checkpoint roundtrip.	$<$ 1 min (CPU)
Stage III	Short-Run Proxy	Multi-seed stability, layer-wise update-to-weight ratio bounds ($R_{t,l} \in [10^{-7}, 0.10]$), loss spike recovery ($F_2$), and 16-bit rounding underflow check ($\Delta\mathcal{L}_{\text{stall}} < 0.25$ nats, F_3).	Minutes (1 GPU)
Stage IV	Large-Scale Run	Train–validation deduplication, low-precision BF16/FP16 stability, isolated optimizer VRAM (VRAM_{opt}), median step time, throughput, and Two-Class held-out loss/PPL/BPB checks.	Hours – Days (GPU / Cluster)
V-Eval	Vision Benchmarks	Track V-A (from-scratch classification & flow matching) and Track V-B (downstream detection & fine-grained transfer) across 6 optimizer setups at Standard or Extended scale.	Hours – Days (GPU / Cluster)
D-Eval	Version Regression	4-check comparison verifying a new version v_k against the previous version v_{k-1}.	Sec – Hours (Cross-Stage)

2 Stage I: Algebraic Safety & Numerical Precision Checks

Before running a single training step, Stage I verifies the mathematical soundness and finite-precision behavior of an update rule. We express the update equations symbolically to prove algebraic

invariants with an SMT solver, then test finite-precision approximations and rounding behavior numerically.

2.1 General SMT Safety Checklist (Z3 Solver)

An update rule $u_t(\theta, g, m, v)$ must not produce NaNs, infinities, or division by zero under valid hyperparameter settings. Stage I verifies seven algebraic conditions using the Z3 Theorem Prover [1] by testing the **negation** of each property and confirming that the solver returns `UNSAT` (unsatisfiable):

Q1. No Division by Zero: For all real gradients $g \in \mathbb{R}$, momentum states $m \in \mathbb{R}$, non-negative second-moment states $v \geq 0$, and $\epsilon > 0$, prove that the update denominator $\mathcal{E}$ (or variance divisor σ) remains strictly positive ($\mathcal{E} > 0$). Negated query: $\mathcal{E} \leq 0$.

Q2. Update Magnitude & Trust-Region Bounds: Confirm that the normalized update u_t and any optional step clipping satisfy the bounds of the optimizer's category:

- *Profile A (Normalized / Bounded-Update Optimizers, e.g., Lion [2] and bounded-ratio designs):* Unit-bounded update magnitude $|u_t| \leq 1.0$, positive coordinate clipping limit $B_i > 0$, and radial clipping lower bound $R \geq r_{\text{bound}}$ [3] when adaptive clipping is enabled.
- *Profile B (Variance-Adaptive Optimizers, e.g., AdamW [4], AdaBelief [5], ADOPT [6]):* Lower-clamped variance divisor $\sigma_{\text{clamped}} \geq \epsilon_{\min} > 0$ and finite step size as $g \to 0$.
- *Profile C (Matrix-Preconditioned / Subspace Optimizers, e.g., Shampoo [7], GaLore [8]):* Bounded matrix operator norm $\|U_t\|_2 \leq C$ and positive minimum singular value $\sigma_{\min}(P) > 0$ for the preconditioner.

Q3. Radial Scaling & Alignment Mask Bounds: When radial norm scaling $S(r)$ is applied for $r \geq 0$, prove that $0 < S(r) \leq 1.0$. When cautious gradient-alignment masking [9] is used with mask $\mathcal{M}_t = \mathbb{I}(u_t \odot g_t > 0)$ and density floor $c_{\min} \in (0, 1]$, prove that the rescaling factor $s_{\text{cautious}} = 1/\text{clip}(\text{mean}(\mathcal{M}_t), c_{\min}, 1.0)$ stays within $[1.0, 1/c_{\min}]$.

Q4. Half-Precision Floor & Overflow Bounds: Under 16-bit execution (`float16` or `bfloat16`), verify that any lower clipping floor stays at or above the IEEE 754 FP16 minimum positive normal threshold ($\text{floor}_{\text{sub}} = 2^{-14} \approx 6.1035 \times 10^{-5}$) and that any FP16 second-moment state is clamped below the FP16 maximum representable value ($65,504.0$).

Q5. Descent Alignment: Prove that starting from zero momentum ($m_0 = 0$) for any non-zero gradient $g_t \neq 0$, the initial update satisfies $\langle u_0, g_t \rangle > 0$, and that under cautious alignment masking [9] ($\mathcal{M}_t = \mathbb{I}(u_t \odot g_t > 0)$), the gradient-driven update prior to decoupled weight decay satisfies $\langle \delta_{\text{cautious}}, g_t \rangle \geq 0$ (with strict positivity whenever $\mathcal{M}_t \neq 0$).

Q6. Weight Decay Contraction: For learning rate $\eta \in (0, 0.05]$, decoupled weight decay [4] $\lambda_{\text{wd}} \in [0, 1.0]$, and effective shrinkage $\eta\lambda_{\text{wd}} < 1.0$, prove that the multiplicative decay factor satisfies $(1 - \eta\lambda_{\text{wd}}) \in (0, 1]$.

Q7. Bias Correction Positivity: For all steps $t \geq 1$ and decay rates $\beta_1, \beta_2 \in (0, 1)$ (encoded via inductive step bounds $\hat{\beta}_{1,t} \in [1 - \beta_1, 1)$ and $\hat{\beta}_{2,t} \in [1 - \beta_2, 1)$), prove that first- and second-moment bias-correction factors remain strictly positive ($\text{bc}_1 = \hat{\beta}_{1,t} > 0, \text{bc}_2 = \hat{\beta}_{2,t} > 0$).

2.2 Symbolic Limits, Function Approximation & Rounding Checks

Alongside Z3 verification, Stage I includes four analytical and finite-precision evaluations:

S1. Large-Gradient Limit Check: Compute the symbolic limit of the normalized update u_t in SymPy [10] as the gradient magnitude grows large along a coupled update step:

$$u_\infty = \lim_{|g|\to\infty} u_t(m, g). \tag{1}$$

Acceptance Rule: Profile A optimizers must satisfy $|u_\infty| \leq 1.0$. Profile B and C optimizers (evaluating coupled second-moment updates or active step clipping for lagged-variance rules [6]) must remain finite without numerical overflow.

S2. Piecewise Transition Continuity (C^0/C^1): When an algorithm switches between piecewise expressions at a small cutoff r_{thresh} (such as substituting 1.0 for $\tanh(r)/r$ near zero), verify that the C^0 boundary value jump Δ_{jump} is smaller than single-precision machine epsilon ($\epsilon_{\text{FP32}} \approx 1.19 \times 10^{-7}$) with bounded $\mathcal{O}(r_{\text{thresh}})$ first-derivative transition.

S3. Function Approximation Accuracy (When Applicable): Kernels that replace standard math functions (tanh, erf, or $1/\sqrt{x}$) with rational or polynomial approximations [11] for SIMD or GPU execution must be evaluated across their input range (500,000 test points) against IEEE 754 `float32`. **Acceptance Rule:** The maximum absolute error must stay below $\|f_{\text{approx}}(x) - f_{\text{exact}}(x)\|_\infty < 1.5 \times 10^{-6}$, while preserving odd/even symmetry and NaN propagation.

S4. Unbiased Stochastic Rounding (When Applicable): Optimizers that write parameters or states into 16-bit or 8-bit tensors via stochastic rounding [12, 13] are tested with $N = 200,000$ samples across both normal ($[10^{-3}, 10^{-1}]$) and FP16 subnormal ($[10^{-5}, 6.1 \times 10^{-5}]$) magnitudes. Practical unbiasedness ($H_0 : |\mathbb{E}[\mathcal{Q}_{\text{SR}}(x)] - x| \geq \delta_{\text{tol}}(x)$) is evaluated with Two One-Sided Tests (TOST) at equivalence tolerance $\delta_{\text{tol}}(x) = \max(10^{-6}, 2.5 \times 10^{-3}\,\text{ULP}(x))$ alongside Monte Carlo standard-error bounds. **Acceptance Rule:** Sample mean bias must satisfy $|\mu_{\mathcal{Q}} - x| < \delta_{\text{tol}}(x)$ with $p_{\text{TOST}} < 0.01$ and standardized error $|(\mu_{\mathcal{Q}} - x)/\text{SE}| < 3.0$ (or $|\mu_{\mathcal{Q}} - x| \leq 0.1\,\text{ULP}(x)$ in subnormal regimes).

3 Stage II: Synthetic Test Functions, Backend Consistency, and Checkpointing

Stage II evaluates optimization trajectories on low-dimensional analytical surfaces, measures sensitivity to hyperparameter perturbations, verifies numerical consistency across CPU, GPU, and sharded multi-GPU backends, and validates checkpoint serialization.

3.1 Synthetic Benchmark Functions

Candidate update rules are compared against standard baselines (such as AdamW [4] in FP32 Master and Native BF16 modes) across five classical test functions [14] using the runner in Listing 1:

B1. Ill-Conditioned Quadratic Bowl ($\kappa \geq 1000$):

$$f(w_0, w_1) = \frac{1}{2}\left(w_0^2 + 1000 w_1^2\right), \qquad w^{(0)} = [10.0, 1.0]^\top. \tag{2}$$

Purpose: Checks whether the optimizer dampens oscillations along the steep axis w_1 without stalling along the flat axis w_0.

B2. Non-Convex Rosenbrock Curved Valley [14, 15]:

$$f(w_0, w_1) = (1 - w_0)^2 + 100(w_1 - w_0^2)^2, \qquad w^{(0)} = [-1.5, 2.0]^\top. \tag{3}$$

Purpose: Evaluates how well the optimizer follows a curved valley where the gradient direction changes continuously.

B3. Convergence Under Gradient Noise: Runs the quadratic bowl with additive Gaussian noise $\tilde{g}_t = \nabla f(w_t) + \mathcal{N}(0, \sigma^2 I)$ at $\sigma = 1.0$. **Purpose:** Measures steady-state error near the optimum when gradients are noisy.

B4. Multimodal Rastrigin Local-Minima Escape [16]:

$$f(w_0, w_1) = 20 + \sum_{i=0}^{1} \left(w_i^2 - 10\cos(2\pi w_i) \right), \qquad w^{(0)} = [2.5, 2.5]^\top. \tag{4}$$

Purpose: Tests whether momentum helps parameters move past shallow local minima toward the global minimum at $(0, 0)$.

B5. High-Dimensional Saddle Point Escape ($N \geq 100$):

$$f(w) = \frac{1}{2}\sum_{i=1}^{N} \lambda_i w_i^2 + \frac{1}{4}\sum_{i=1}^{N} w_i^4, \qquad f^* = -\frac{1}{4}\sum_{\lambda_i < 0} \lambda_i^2, \\ \lambda_i \in [-1.0, 1000.0], \qquad w_i^{(0)} = 10^{-4}. \tag{5}$$

Purpose: Measures how quickly the optimizer escapes saddle points at the origin combining negative and large positive eigenvalues (spaced linearly via `np.linspace(-1.0, 1000.0, N)`) toward the global minimum $f^* < 0$ (attained at $w_i^* = \pm\sqrt{-\lambda_i}$ for $\lambda_i < 0$ and $w_i^* = 0$ for $\lambda_i \geq 0$).

Listing 1: Standard LSO-Eval Stage II: General Synthetic Function Test Runner (v1.1.1)

```python
import numpy as np

def run_synthetic_benchmark(loss_fn, grad_fn, step_fn, w_init, f_star=0.0,
                            lr=0.01, max_steps=2000, tol=1e-5,
                            noise_std=0.0, seed=1337):
    rng = np.random.default_rng(seed)
    w = np.array(w_init, dtype=np.float64)
    state = {}
    current_loss = loss_fn(w)

    for t in range(1, max_steps + 1):
        g = grad_fn(w)
        if noise_std > 0.0:
            g = g + rng.normal(0.0, noise_std, size=g.shape)
        w, state = step_fn(w, g, state, lr, t)
        current_loss = loss_fn(w)
        curr_gap = abs(current_loss - f_star)
        if np.isnan(w).any() or np.isinf(w).any() or curr_gap > 1e8:
            return t, curr_gap, "DIVERGED"
        if curr_gap < tol:
            return t, curr_gap, "CONVERGED"
    return max_steps, abs(current_loss - f_star), "MAX_STEPS"
```

3.2 Hyperparameter Sensitivity Checks

Practical optimizers must converge reliably without brittle hyperparameter tuning [4]. We evaluate robustness against three criteria:

P1. One-at-a-Time Parameter Sweep: Keep default settings fixed and vary each hyperparameter across at least 5 values spanning an order of magnitude.

P2. Analytical Parameter Bounds: Whenever a coefficient controls a stability condition or balances competing update terms, derive its valid range analytically where possible and validate inputs when initializing the optimizer.

P3. Parameter Stability Range: Perturbing auxiliary hyperparameters by ±30% around their defaults must not increase the convergence step count on the $\kappa = 1000$ quadratic benchmark by more than 15%.

3.3 Feature Combinations, Single-Pass Reductions & Backend Consistency

Multi-backend optimizer libraries must produce consistent results across configuration flags, hardware targets, and sharded multi-GPU modes:

A1. Feature Combination Stability: Every supported combination of state buffer modes, step-clipping options [3], and alignment masks [9] must produce finite, convergent updates.

A2. Single-Pass Reduction Consistency (When Applicable): Kernels that fuse global norm or mask reductions into a single memory pass [17, 18] by reading exponential moving average (EMA) statistics from step $t-1$ must converge within +1.0% of the step count of an exact two-pass reduction on the $\kappa = 1000$ benchmark.

A3. Cross-Backend & Sharded Parity:

- *Parallel Reduction Parity:* Multithreaded CPU reductions must use deterministic pairwise or tree summation so outputs match single-threaded scalar execution (atol = 0.0).
- *PRNG & Rounding Parity:* Vectorized SIMD PRNG and stochastic rounding routines must match the scalar reference bit-for-bit (atol = 0.0).
- *Multi-Step Update Parity ($N = 131,072$, 5 steps):* Across supported floating-point formats (FP32, BF16, FP16), vectorized CPU and GPU compute backends must agree with the scalar reference within atol $\leq 10^{-5}$ for FP32 and atol $\leq 2 \times 10^{-2}$ for BF16/FP16.
- *Sharded vs. Unsharded Parity (FSDP / ZeRO):* When an optimizer computes tensor-wide or layer-wise reductions (such as cautious alignment mask density $\bar{m}_t$ [9] or radial norm clipping [3]) under Fully Sharded Data Parallel (FSDP) or ZeRO parameter sharding, local reduction sums must be synchronized across ranks via `all_reduce` so sharded parameter updates match single-device unsharded updates within atol $\leq 10^{-6}$ (FP32).
- *Python Binding Parity:* NumPy and PyTorch bindings must produce identical parameter tensors when given identical inputs and seeds.

3.4 Checkpoint Saving and Loading Check

Subtle state-serialization bugs can silently corrupt resumed training runs or bloat checkpoints with temporary workspace buffers. Every candidate must pass the **Checkpoint Roundtrip Test** (Listing 2) on both `float32` and `bfloat16` models. The test enforces three guarantees: (1) temporary coalescing or staging buffers are stripped prior to `state_dict()` export without corrupting the live optimizer instance, (2) per-step scratch accumulators are zeroed out on save while `float32` master weights and scratchpads retain their exact `float32` dtype upon `load_state_dict()` (preventing PyTorch's default state-casting from downcasting them to `bfloat16`), and (3) training resumed from a checkpoint matches an uninterrupted run bit-for-bit (atol = 0.0).

Listing 2: Standard LSO-Eval Stage II: Checkpoint Roundtrip & State Cleanup Test (v1.1.1)

```
import copy, io, torch, torch.nn as nn

```

```
def verify_checkpoint_integrity(optimizer_cls, opt_configs=None,
                                forbidden_keys=(), zeroed_scratch_indices=(0, 1, 2)):
    torch.manual_seed(42)
    if opt_configs is None:
        opt_configs = [{"lr": 1e-3}]
    default_forbidden = (
        "p_contig", "exp_avg_contig", "exp_avg_sq_contig",
        "mp_contig", "_plan", "_contig_step"
    )
    all_forbidden = set(default_forbidden).union(forbidden_keys)

    for dtype in (torch.float32, torch.bfloat16):
        for cfg in opt_configs:
            model_cont = nn.Sequential(
                nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 2)
            ).to(dtype=dtype)
            model_save = copy.deepcopy(model_cont)
            model_res = copy.deepcopy(model_cont)
            opt_cont = optimizer_cls(model_cont.parameters(), **cfg)
            opt_save = optimizer_cls(model_save.parameters(), **cfg)
            batches = [
                (torch.randn(4, 10, dtype=dtype), torch.tensor([0, 1, 0, 1]))
                for _ in range(6)
            ]

            for i in range(5):
                x, y = batches[i]
                rng_snap = torch.get_rng_state()
                for m, opt in ((model_cont, opt_cont), (model_save, opt_save)):
                    torch.set_rng_state(rng_snap)
                    loss = nn.CrossEntropyLoss()(m(x).float(), y)
                    opt.zero_grad(); loss.backward(); opt.step()

            model_res.load_state_dict(copy.deepcopy(model_save.state_dict()))
            buffer = io.BytesIO()
            torch.save(opt_save.state_dict(), buffer); buffer.seek(0)
            opt_loaded = optimizer_cls(model_res.parameters(), **cfg)
            state_dict = torch.load(buffer, weights_only=False)
            opt_loaded.load_state_dict(state_dict)

            for p_state in state_dict["state"].values():
                for k in all_forbidden:
                    assert k not in p_state, f"Leaked key '{k}' in state_dict!"
                if "scratch" in p_state and zeroed_scratch_indices:
                    sc = p_state["scratch"]
                    for idx in zeroed_scratch_indices:
                        assert float(sc[idx]) == 0.0

            for p in model_res.parameters():
                st = opt_loaded.state[p]
                if "scratch" in st:
                    assert st["scratch"].dtype == torch.float32
                if "master_param" in st:
                    assert st["master_param"].dtype == torch.float32

            x_last, y_last = batches[5]
            rng_snap = torch.get_rng_state()
            for m, opt in ((model_cont, opt_cont), (model_save, opt_save), (model_res,
                opt_loaded)):
                torch.set_rng_state(rng_snap)
                loss = nn.CrossEntropyLoss()(m(x_last).float(), y_last)
                opt.zero_grad(); loss.backward(); opt.step()

            for p_c, p_s, p_r in zip(model_cont.parameters(), model_save.parameters(),
                model_res.parameters()):
                torch.testing.assert_close(p_c, p_s, rtol=0.0, atol=0.0)
                torch.testing.assert_close(p_c, p_r, rtol=0.0, atol=0.0)
    return True
```

4 Stage III: Short-Run Proxy Model Screening

Stage III evaluates candidate optimizers on a compact neural network proxy before committing to multi-hour GPU experiments. This screening phase filters out unstable learning rates and catches low-precision mantissa underflow early.

4.1 Proxy Model Selection, Multi-Seed Protocol & Step Budget

Screening runs use a small proxy model sharing the core building blocks of the target architecture (for instance, a 4-layer causal Transformer using RoPE [19], SwiGLU MLP layers, and RMSNorm [20]):

M1. Step Limit: Runs are budgeted at 5% to 15% of a full training schedule (typically 500 to 2,000 steps).

M2. Batch Size Diversity: Each candidate is tested at two effective batch sizes (1× micro-batch and 4× gradient-accumulated batch) to verify stability across different gradient noise regimes [6].

M3. Matched Schedule & Gradient Clipping: All tested optimizers share an identical schedule: linear warmup over the first 5% of steps followed by cosine decay (or Warmup-Stable-Decay, WSD) to 5% of the peak learning rate. If external global gradient norm clipping (`max_grad_norm`) is used, the same threshold (e.g., 1.0) must be applied before `opt.step()` across all runs.

M4. Multi-Seed Statistical Reporting: Because short proxy runs take only minutes, each configuration is evaluated across at least 3 random seeds (e.g., seed $\in \{42, 1337, 2026\}$), reporting mean $\pm$ std (or median) final loss alongside the percentage of unstable or diverged trials.

4.2 Early Stopping & Rejection Criteria

A candidate configuration fails Stage III if it triggers any of three failure conditions:

F1. Out-of-Range Update-to-Weight Ratio: Track the layer-wise relative parameter step norm across weight matrices with ndim ≥ 2 (excluding singleton-broadcasted vectors with ndim > 2 and `param.squeeze().ndim` ≤ 1):

$$R_{t,l} = \frac{\|\Delta\theta_{t,l}\|_2}{\max(\|\theta_{t,l}\|_2, 10^{-3})}, \qquad \Delta\theta_{t,l} = \theta_{t,l} - \theta_{t-1,l}. \tag{6}$$

This check measures the committed post-update parameter difference $\Delta\theta_{t,l}$, omits 1D biases, normalization scales, and singleton-broadcasted vectors, and logs backbone matrices separately from token embeddings and the final output head [3]. **Rejection Rule:** A run is rejected if $R_{t,l} > 0.10$ on any matrix layer l after warmup (step size too large) or if $R_{t,l} < 10^{-7}$ during active training ($t > t_{\text{warmup}}$ and $\eta_t \geq 0.1\eta_{\max}$, indicating stalled updates).

F2. Unrecovered Loss Spikes: If the exponential moving average (EMA, $\alpha = 0.99$) training loss rises by more than 0.5 nats above its prior minimum and fails to recover below that threshold (or recover at least 50% of the spike excursion) within 50 consecutive steps, the trial is rejected.

F3. BF16/FP16 Rounding Underflow & Update Cancellation: Compare the final EMA training loss of the native 16-bit (`bfloat16` or `float16`) trial against a matched reference run in `float32` precision [12, 13]:

$$\Delta\mathcal{L}_{\text{stall}} = \mathcal{L}_{\text{EMA}}^{(\text{BF16/FP16})}(T) - \mathcal{L}_{\text{EMA}}^{(\text{FP32-Ref})}(T). \tag{7}$$

Following Zamirai et al. [13], implementations may also log the *16-bit Update Cancellation Rate*—the fraction of non-zero updates $\delta_{t,i} \neq 0$ where $\theta_{t+1,i} == \theta_{t,i}$ after 16-bit addition because $|\Delta\theta_{t,i}| = |\eta_t \delta_{t,i}| < \frac{1}{2}\text{ULP}(\theta_{t,i})$. **Rejection Rule:** A loss gap of $\Delta\mathcal{L}_{\text{stall}} \geq 0.25$ nats indicates significant mantissa truncation and results in rejection.

4.3 Learning Rate & Clipping Grid

Candidates are swept across a learning rate grid ($\eta \in [3 \times 10^{-4}, 10^{-2}]$) covering two training regimes:

- **Long-Run Setting ($>$ 20k steps):** Moderate learning rates ($\eta \in [3 \times 10^{-4}, 8 \times 10^{-4}]$, $\lambda_{\text{wd}} = 8 \times 10^{-2}$) intended for extended pre-training schedules.
- **Short-Run Setting ($<$ 10k steps):** Higher learning rates ($\eta = 10^{-2}$) paired with coordinate or radial step clipping [3].

5 Stage IV: Large-Scale Pre-Training Evaluation

Stage IV benchmarks candidate optimizers on language model pre-training across two schedule lengths and two hardware scale tiers:

1. **Stage IV-A (Medium-Length Run):** Runs of $2,500$ to $5,000$ steps at sequence length 512 with gradient accumulation (reference: $8\times$, $4,096$ tokens/step), measuring isolated optimizer VRAM, median step time, token throughput, and held-out validation loss and perplexity.
2. **Stage IV-B (Full-Schedule & Extended-Context Run):** Longer training runs ($\geq 20,000$ steps at sequence length $\geq 2,048$) testing convergence through full cosine or Warmup-Stable-Decay (WSD) learning rate decay.

Standard Setup (Single-GPU) vs. Extended Setup (Multi-GPU / Cluster): To keep LSO-Eval accessible to single-GPU researchers while scaling cleanly to multi-node clusters, Stage IV defines two validation tiers:

- **Standard Setup (Single-GPU):** Uses a deduplicated held-out validation split of $N_{\text{eval}} \geq 450,000$ tokens (reference default: $880 \times 512 = 450,560$ tokens in Stage IV-A and $220 \times 2,048 = 450,560$ tokens in Stage IV-B) with a matched deterministic seed (`seed=42`).
- **Extended Setup (Multi-GPU / Cluster):** Multi-node runs spanning 10B to 350B+ tokens [9, 21] use a two-tier validation schedule. During training, periodic validation runs on 450k–5M deduplicated tokens every N steps to monitor loss curves with minimal overhead. At the end of training, models are evaluated on 10M–100M+ held-out tokens across diverse text and code corpora (such as Paloma [22], C4, WikiText-103, and PG-19 [23]), paired with zero-shot downstream benchmarks (`MMLU`, `ARC`, `HellaSwag`, `PIQA`, `WinoGrande`, `OpenBookQA`, and `LAMBADA`) [9, 21, 23].

5.1 Data Deduplication, Domain Mixing, and Sequence Packing

Streaming pre-training benchmarks require clean data preparation so validation overlap, domain block ordering, and padding waste do not distort comparisons across optimizers:

D1. Deterministic Document Hash Logging: Every document included in the held-out validation split ($N_{\text{eval}} \geq 450,000$ tokens in Standard Setup; $\geq$ 10M tokens in Extended Setup) is hashed using a fast 64-bit non-cryptographic hash (reference default: 64-bit XXH3 recorded in a persistent `.dedup` hash log file).

D2. Cross-Context Token Multiset Check: When validation is configured across different sequence lengths (for example, $880 \times 512 = 450,560$ tokens in Stage IV-A and $220 \times 2,048 = 450,560$ tokens in Stage IV-B), the sorted token multisets are verified to be identical so evaluation corpora remain strictly matched.

D3. Stream Offset & Hash Filtering: Training data streams advance past the raw records reserved for validation (`skip_raw_items`) and load the validation hash set prior to step 1, dropping any training document whose hash matches the validation set [21].

D4. Weighted Multi-Domain Interleaving (No Block Concatenation): When pulling from K text domains $D_1, \ldots, D_K$ with mixture weights $\mathbf{p} = (p_1, \ldots, p_K)$ ($\sum_{k=1}^{K} p_k = 1$), streams are combined via seeded weighted interleaving (such as multinomial sampling or windowed quotas) rather than concatenating datasets sequentially. This keeps the domain mixture stationary across warmup, peak learning rate, and decay.

D5. Two-Step Streaming Shuffle: Streaming loaders apply *file-level shuffling* (permuting shard order via `seed + epoch`) followed by *document-level shuffling* in an in-memory reservoir buffer ($B_{\text{shuf}} \geq 10,000$ documents, filled before step 1) so adjacent documents within a packed sequence come from different shards and domains.

D6. EOD-Delimited Sequence Packing & Asynchronous Transfer: Instead of padding short documents with `<PAD>` tokens, tokenized documents are terminated with an explicit End-of-Document token (`EOD`/`EOS`), concatenated into a continuous stream, and sliced into dense (B, L) blocks with 100% token utilization. When using variable-length FlashAttention [17], loaders pass cumulative sequence offsets (`cu_seqlens`) and reset position indices per document. Batches are staged in page-locked host memory and transferred asynchronously to the GPU.

5.2 Measuring Optimizer Memory, Step Time, and Runtime Environment

All candidate and baseline runs must execute on identical hardware with matched seeds, batch order, and context lengths. To keep timing and memory numbers reproducible, every Stage IV report must document the hardware and software environment: GPU model, interconnect, and VRAM capacity, GPU driver and compute runtime versions (CUDA, ROCm, or Vulkan), framework and compiler versions (e.g., PyTorch, Triton, Rust/C++ toolchain), and the global random seed(s).

Following prior work on memory-efficient optimizers (8-bit Optimizers [24] and GaLore [8]), Stage IV separates peak active tensor memory (`torch.cuda.max_memory_allocated()`) into the fixed model/activation baseline $\text{VRAM}_{\text{base}}$ and the **Isolated Optimizer Memory** VRAM_{opt} (Table 2):

$$\text{VRAM}_{\text{peak}} = \underbrace{W_{\text{model}} + G_{\text{grad}} + A_{\text{ctx}}}_{\text{Fixed Model \& Activation Baseline } (\text{VRAM}_{\text{base}})} + \underbrace{\text{VRAM}_{\text{opt}}}_{\text{Isolated Optimizer State \& Master Copy}}. \quad (8)$$

Because `max_memory_allocated()` measures active tensor bytes allocated by PyTorch (excluding cached allocator pools and runtime context) and $\text{VRAM}_{\text{base}}$ stays constant for a fixed model and sequence length, the difference $\text{VRAM}_{\text{opt}} = \text{VRAM}_{\text{peak}} - \text{VRAM}_{\text{base}}$ (alongside peak allocation isolated across `opt.step()` after forward activations A_{ctx} are released) isolates the memory consumed by optimizer state buffers, transient workspaces, and any master weight copies [8, 24].

5.3 Two-Class Acceptance Criteria

Validation checkpoints are scored by held-out cross-entropy loss $\mathcal{L}_{\text{val}}$ (in nats), perplexity $\text{PPL}_{\text{held-out}}$, and (when comparing across tokenizers) Bits-Per-Byte (BPB):

$$\mathcal{L}_{\text{val}} = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \mathcal{L}_{\text{CE}}(x_i), \qquad \text{PPL}_{\text{held-out}} = \exp(\mathcal{L}_{\text{val}}), \qquad \text{BPB} = \frac{N_{\text{eval}}}{B_{\text{UTF8}} \ln 2} \mathcal{L}_{\text{val}}, \quad (9)$$

Table 2: Stage IV: Standard Evaluation Metrics (v1.1.1)

Dimension	Metric	Measurement Method & Reporting Standard
1. Convergence	Raw Loss, $\mathcal{L}_{\text{EMA}}$ & Steps-to-Target	Logged at every step; smoothed via EMA ($\alpha \in \{0.99, 0.995\}$). Reports steps and wall-clock time needed to reach the final baseline validation loss.
2. Generalization	Val Loss ($\mathcal{L}_{\text{val}}$), Val PPL & BPB	Evaluated on $\geq$ 450k deduplicated tokens (Standard) or 10M–100M+ multi-domain tokens (Extended): $\text{PPL} = \exp(\mathcal{L}_{\text{val}})$ and $\text{BPB} = \frac{N_{\text{eval}}}{B_{\text{UTF8}} \ln 2}\mathcal{L}_{\text{val}}$. Optional zero-shot downstream accuracy (`MMLU`, `ARC`, `HellaSwag`, `PIQA`) in Stage IV-B.
3. Memory	$\text{VRAM}_{\text{peak}}$ & VRAM_{opt} (GB)	Active tensor peak via `max_memory_allocated()` [25] and isolated $\text{VRAM}_{\text{opt}} = \text{VRAM}_{\text{peak}} - \text{VRAM}_{\text{base}}$ [8, 24].
4. Latency & Speed	Median Step Time, Opt Time & Tok/s	Median total step time (ms) [3], isolated `opt.step()` kernel time (ms and normalized ms/1B params [24]), and throughput (tokens/sec [8]).

where B_{UTF8} is the total UTF-8 byte length of the held-out evaluation text. Because single-buffer (≈ 2 B/param) and dual-buffer (4–12 B/param) optimizers target different hardware memory budgets [8, 24], LSO-Eval v1.1.1 defines **Two-Class Acceptance Criteria** measured against Native BF16 AdamW [4] and FP32 Master AdamW:

Class I. Dual-Buffer / Full-State Optimizers (≥ 4 bytes/param): Optimizers that store both first- and second-moment buffers (m_t, v_t) pass Class I if they achieve **equal or lower held-out validation loss and perplexity** than Native BF16 AdamW at the same token budget ($\Delta\mathcal{L}_{\text{val}} \leq 0.0$ and $\Delta\text{PPL}_{\text{held-out}} = \text{PPL}_{\text{cand}} - \text{PPL}_{\text{AdamW-BF16}} \leq 0.0$), satisfy $F_1 \wedge F_2$ stability bounds, keep median isolated `opt.step()` time within 1.15× of Native BF16 AdamW, and require no extra optimizer memory ($\Delta\text{VRAM}_{\text{opt}} \leq 0.0$ GB at 0.01 GB resolution).

Class II. Single-Buffer / Compressed-State Optimizers (≤ 2 bytes/param): Optimizers that drop the second-moment buffer v_t (or compress state tensors) to fit larger models on limited VRAM pass Class II if they meet four conditions:

(a) *Update Ratio and Loss Stability ($F_1 \wedge F_2$):* Layer-wise update ratios stay within $R_{t,l} \in [10^{-7}, 0.10]$ with no unrecovered loss spikes.

(b) *Consistent Validation Improvement:* Held-out validation loss and perplexity improve across evaluation checkpoints without late-stage divergence.

(c) *Memory & Step Efficiency:* Cuts isolated optimizer memory by at least 35.0% ($\Delta\text{VRAM}_{\text{opt}} \leq -35.0\%$) relative to Native BF16 AdamW and at least 70.0% ($\Delta\text{VRAM}_{\text{opt}} \leq -70.0\%$) relative to FP32 Master AdamW[1], while keeping median isolated `opt.step()` time within 1.15× of Native BF16 AdamW and faster than FP32 Master AdamW.

[1]For a pure single-buffer `bfloat16` state (2 B/param) on multi-billion-parameter models where fixed allocator rounding is negligible, the exact state savings reach −50.0% vs. Native BF16 AdamW (4 B/param) and −83.3% vs. FP32 Master AdamW (12 B/param). The −35.0% and −70.0% thresholds leave room for allocator block alignment on smaller models and hybrid or subspace compression schemes.

(d) *Bounded Compression Loss Gap:* The validation log-perplexity gap relative to two-buffer Native BF16 AdamW stays within the Stage III F_3 bound:

$$\Delta\mathcal{L}_{\text{val}} = \ln\left(\frac{\text{PPL}_{\text{cand}}}{\text{PPL}_{\text{AdamW-BF16}}}\right) \leq 0.25 \text{ nats.} \tag{10}$$

6 V-Eval: Vision Model Benchmarks

Autoregressive language models only tell part of the optimization story. In computer vision, convolutional stems, batch and global response normalization, scaled weight standardization, heavily augmented Vision Transformers, and flow-matching generative models place very different demands on momentum dynamics and step clipping. **V-Eval** complements the language modeling stages with a dedicated vision suite spanning multiple spatial resolutions.

6.1 Model Sources, Two-Track Structure, and Scale Setups

All V-Eval benchmarks use standard model definitions from open-source libraries—`timm` (PyTorch Image Models), `torchvision.models`, Hugging Face `transformers`, and the official `facebookresearch/SiT` repository.

V-Eval is organized into two tracks across two hardware scale setups:

1. **Track V-A (Core — Vision From-Scratch Training):** Trains five vision backbones (V-A1 through V-A5) from random initialization (`pretrained=False`, `seed=42`, or ≥ 3 seeds when compute permits) in native `torch.bfloat16` across three image resolutions (32×32, 128×128, and 224×224).
2. **Track V-B (Optional — Vision Downstream Fine-Tuning):** Fine-tunes pretrained backbones on object detection (V-B1) and fine-grained classification (V-B2) when extra compute is available. Track V-B tests whether an optimizer preserves pretrained features without suffering BF16 rounding underflow at small learning rates ($\eta \leq 10^{-4}$).

Standard Setup (Single-GPU) vs. Extended Setup (Multi-GPU / Cluster) in Vision:

- **Standard Setup (Single-GPU Default, Tables 3–4):** Uses **CIFAR-100** (50,000 train / 10,000 val images at 32^2), **ImageWoof** (9,469 train / 3,925 val images at 224^2), **Mini-ImageNet** (50,000 train / 10,000 val images at 128^2), **CIFAR-10 flow matching** (50,000 train / 10,000 val images at 32^2, evaluated with FID@10k), **cppe-5** or **Pascal VOC 2007** (480^2) for detection, and **Oxford-IIIT Pet** (3,680 train / 3,669 test images at 224^2) for fine-grained transfer.
- **Extended Setup (Multi-GPU / Cluster):** For multi-GPU or TPU clusters [2, 3, 6], implementations may additionally run from-scratch and fine-tuning benchmarks on **Full ImageNet-1K** (1.28M train / 50k val images across 1,000 classes at 224^2–384^2) or **ImageNet-21K**. Out-of-distribution (OOD) generalization is tested on `ImageNet-ReaL`, `ImageNet-V2`, `ImageNet-A`, `ImageNet-R`, and `ObjectNet` ($> 190,000$ evaluation images in total) [2], alongside **Full MS-COCO 2017** (118k train / 5k val) for object detection and **FID@50k on ImageNet** 256^2 for generative models [26].

6.2 Candidate and Baseline Optimizers and Parameter Grouping

To evaluate native `bfloat16` training (no master weights) alongside `float32` master-weight training across both two-buffer and single-buffer designs, every task in Track V-A and Track V-B compares six optimizer setups:

1. `Candidate_Class_I` (**Class I Candidate — 2-Buffer Native BF16**): Full-state candidate optimizer ($m_t, v_t, \geq 4$ B/param) executed in native `bfloat16` without `float32` master weights, using decoupled weight decay $\lambda_{\text{wd}} = 0.08$ for from-scratch training and $\lambda_{\text{wd}} = 0.05$ for fine-tuning.
2. `Candidate_Class_II` (**Class II Candidate — 1-Buffer Native BF16**): Single-buffer or compressed-state candidate optimizer (≤ 2 B/param) executed in native `bfloat16` without `float32` master weights, using decoupled weight decay $\lambda_{\text{wd}} = 0.08$ for from-scratch training and $\lambda_{\text{wd}} = 0.05$ for fine-tuning.
3. `AdamW_Native_BF16` (**Baseline 2-Buffer — Native BF16**): `torch.optim.AdamW` [4] configured with `betas=(0.9, 0.999)`, `eps=1e-8`, `weight_decay=0.08` (From-Scratch) / `0.05` (Fine-Tuning), and `fused=True`, executed directly on `bfloat16` parameters (`use_fp32_master=False` in the benchmark harness).
4. `AdamW_FP32_Master` (**Reference 2-Buffer — BF16 + FP32 Master Copy**): Identical to `AdamW_Native_BF16`, wrapped with an explicit `float32` master weight copy in the harness (`use_fp32_master=True`).
5. `Lion_Native_BF16` (**Baseline 1-Buffer — Native BF16**): `SimpleLion` [2] (Listing 3), `betas=(0.9, 0.99)`, `weight_decay=0.8` (matching effective shrinkage $\eta\lambda = 8 \times 10^{-5}$ in Track V-A), `use_fp32_master=False`.
6. `Lion_FP32_Master` (**Reference 1-Buffer — BF16 + FP32 Master Copy**): Identical to `Lion_Native_BF16`, wrapped with an explicit `float32` master weight copy in the harness (`use_fp32_master=True`).

Weight Decay Parameter Grouping (`split_vision_parameter_groups`): Across all six optimizers in V-Eval (as well as LLM and hybrid runs in Stages III and IV), model parameters are partitioned into two groups before training (see Listing 3 for the complete keyword list):

- **Decayed Group ($\lambda_{\text{wd}} > 0$):** All weight matrices and convolutional kernels with at least two non-singleton dimensions (ndim ≥ 2, excluding singleton-broadcasted vectors with ndim > 2 and `param.squeeze().ndim` ≤ 1), including standard and depthwise convolutions, linear projections, attention matrices, MLP weights, and token embeddings (unless excluded via `exclude_embeddings=True`).
- **No-Decay Group ($\lambda_{\text{wd}} = 0.0$):**
  (a) *0D/1D and Singleton-Broadcasted Tensors:* Biases, 1D affine scales and shifts, and channel- or head-wise vectors stored with singleton broadcast dimensions (such as $(1, 1, 1, C)$ GRN parameters in ConvNeXt-V2 or $(1, C, 1, 1)$ channel scales where ndim > 2 and `param.squeeze().ndim` ≤ 1).
  (b) *Normalization, Scaling & Temperature Parameters:* Parameters belonging to normalization layers (`BatchNorm`, `LayerNorm`, `RMSNorm` [20], `GroupNorm`, `GRN`, `RaNorm`/`L1Norm`, and attention `QK-Norm`), residual scaling gains (`LayerScale`, Dynamic Tanh `DyT` [27], and NF-Net skip gains [3]), and learnable attention or contrastive temperature scalars (`logit_scale`, `head_scale`).
  (c) *Positional, Register & Query Tables:* Structural tokens (`cls_token`, `reg_token`, `mask_token`), absolute and relative positional embeddings (`pos_embed`, relative bias tables, and rotary frequencies [19]), and object detection query tables (`query_embed`, `query_pos`).
  (d) *Sequence-Model Decay Parameters, MoE Biases & Custom Hooks:* State-space and linear-attention decay parameters in Mamba, GLA [23], and RWKV (`A_log`, `dt_bias`, 1D skip vector `D`, `time_decay`), router bias terms in Mixture-of-Experts models, and any parameter marked via `no_weight_decay()` model hooks.

Baseline Parameter Alignment and Effective Weight Decay (v1.1.1): For vision benchmarks in LSO-Eval v1.1.1, the `SimpleLion` baseline uses $\beta = (0.9, 0.99)$ and decoupled weight decay $\lambda = 0.8$ (Listing 3), following the vision settings and hyperparameter scaling rule in Chen et al. [2] (Section 3.2 and Appendix M). Because decoupled weight decay multiplies weights by $(1 - \eta\lambda)$ at each step and Lion operates at a 10× smaller learning rate ($\eta_{\text{Lion}} = 10^{-4}$) than AdamW ($\eta_{\text{AdamW}} = 10^{-3}$), setting $\lambda_{\text{Lion}} = 0.8$ keeps per-step weight shrinkage identical across baselines:

$$\eta_{\text{Lion}} \cdot \lambda_{\text{Lion}} = 10^{-4} \times 0.8 = 8 \times 10^{-5} = 10^{-3} \times 0.08 = \eta_{\text{AdamW}} \cdot \lambda_{\text{AdamW}}. \tag{11}$$

Listing 3: Standard LSO-Eval V-Eval: Vectorized PyTorch SimpleLion Optimizer (Chen et al., NeurIPS 2023, Algorithm 2) & Decoupled Parameter Grouping

```python
from collections import defaultdict
import torch
import torch.nn as nn

class SimpleLion(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-4, betas=(0.9, 0.99), weight_decay=0.8,
                 foreach=True, use_fp32_master=False):
        if lr < 0.0:
            raise ValueError(f"Invalid learning rate: {lr}")
        if not 0.0 <= betas[0] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 0: {betas[0]}")
        if not 0.0 <= betas[1] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 1: {betas[1]}")
        if weight_decay < 0.0:
            raise ValueError(f"Invalid weight_decay value: {weight_decay}")
        defaults = dict(
            lr=lr, betas=betas, weight_decay=weight_decay,
            foreach=foreach, use_fp32_master=use_fp32_master
        )
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr = group["lr"]
            beta1, beta2 = group["betas"]
            wd = group["weight_decay"]
            use_foreach = group.get("foreach", True)
            use_master = group.get("use_fp32_master", False)
            buckets = defaultdict(lambda: ([], [], [], []))

            for p in group["params"]:
                if p.grad is None:
                    continue
                grad = p.grad
                if grad.is_sparse:
                    raise RuntimeError("SimpleLion does not support sparse grads.")
                state = self.state[p]
                if len(state) == 0:
                    stat_dtype = torch.float32 if use_master else p.dtype
                    state["exp_avg"] = torch.zeros_like(
                        p, dtype=stat_dtype, memory_format=torch.preserve_format
                    )
                    if use_master and p.dtype != torch.float32:
                        state["master_param"] = p.detach().float().clone()
                work_p = state.get("master_param", p)
                work_g = grad.to(dtype=work_p.dtype) if grad.dtype != work_p.dtype else
                    grad
                key = (work_p.device, work_p.dtype)
                buckets[key][0].append(work_p)
                buckets[key][1].append(work_g)
                buckets[key][2].append(state["exp_avg"])
                buckets[key][3].append(p)

```

```
            for (_, _), (work_params, grads, exp_avgs, orig_params) in buckets.items():
                if use_foreach:
                    if wd != 0.0:
                        torch._foreach_mul_(work_params, 1.0 - lr * wd)
                    updates = torch._foreach_lerp(exp_avgs, grads, 1.0 - beta1)
                    torch._foreach_sign_(updates)
                    torch._foreach_add_(work_params, updates, alpha=-lr)
                    torch._foreach_lerp_(exp_avgs, grads, 1.0 - beta2)
                else:
                    for wp, g, m in zip(work_params, grads, exp_avgs):
                        if wd != 0.0:
                            wp.mul_(1.0 - lr * wd)
                        u = m.lerp(g, 1.0 - beta1).sign_()
                        wp.add_(u, alpha=-lr)
                        m.lerp_(g, 1.0 - beta2)
                if use_master:
                    pairs = [(p_o, wp) for p_o, wp in zip(orig_params, work_params) if
                        p_o is not wp]
                    if pairs:
                        if use_foreach:
                            torch._foreach_copy_([x[0] for x in pairs], [x[1] for x in
                                pairs])
                        else:
                            for p_o, wp in pairs:
                                p_o.copy_(wp)
        return loss

    def load_state_dict(self, state_dict):
        super().load_state_dict(state_dict)
        for group in self.param_groups:
            if group.get("use_fp32_master", False):
                for p in group["params"]:
                    st = self.state.get(p, {})
                    for k in ("master_param", "exp_avg"):
                        if k in st and st[k].dtype != torch.float32:
                            st[k] = st[k].to(dtype=torch.float32)

def split_vision_parameter_groups(model: nn.Module, weight_decay: float = 0.08,
                                  exclude_embeddings: bool = False):
    no_decay_substr = (
        "bias", "norm", "rmsnorm", "groupnorm", "ranorm", "l1norm",
        "global_response_norm", "q_norm", "k_norm", "qk_norm", "layerscale",
        "gamma", "beta", "dynamic_tanh", "alpha", "gain", "skipinit",
        "logit_scale", "logit_bias", "head_scale", "cpb_mlp",
        "cls_token", "reg_token", "register_token", "dist_token", "mask_token",
        "pos_embed", "positional_embedding", "class_embedding",
        "absolute_pos_embed", "relative_position_bias_table", "rel_pos",
        "rope", "freqs_cis", "query_embed", "query_pos", "level_embed",
        "a_log", "dt_bias", "d_skip", "time_mix", "time_decay", "time_first",
        "e_score_correction_bias", "expert_bias",
    )
    no_decay_tokens = {"bn", "ln", "grn", "ls1", "ls2", "dyt"}
    if exclude_embeddings:
        no_decay_substr = no_decay_substr + (
            "tok_embeddings", "embed_tokens", "word_embeddings"
        )
        no_decay_tokens.add("wte")

    has_nwd = hasattr(model, "no_weight_decay") and callable(model.no_weight_decay)
    explicit_no_decay = set(model.no_weight_decay()) if has_nwd else set()

    has_kw = (
        hasattr(model, "no_weight_decay_keywords")
        and callable(model.no_weight_decay_keywords)
    )
    keyword_no_decay = (
        tuple(k.lower() for k in model.no_weight_decay_keywords())
        if has_kw else ()
    )

    decay, no_decay = [], []
    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
```

```
        lname = name.lower()
        segments = {
            seg.rstrip("0123456789")
            for seg in lname.replace(".", "_").split("_")
        }
        if (
            param.ndim <= 1
            or (param.ndim > 2 and param.squeeze().ndim <= 1)
            or getattr(param, "_no_weight_decay", False)
            or getattr(param, "no_weight_decay", False)
            or name in explicit_no_decay
            or bool(segments & no_decay_tokens)
            or any(k in lname for k in no_decay_substr)
            or any(k in lname for k in keyword_no_decay)
        ):
            no_decay.append(param)
        else:
            decay.append(param)
    return [
        {"params": decay, "weight_decay": float(weight_decay)},
        {"params": no_decay, "weight_decay": 0.0},
    ]
```

6.3 Track V-A: Vision From-Scratch Evaluation (Core)

Every Track V-A model is trained from random initialization (`pretrained=False`, `seed=42`) in `torch.bfloat16` on a single GPU (reference hardware: 1× NVIDIA A100 40 GB). Evaluating ImageNet-stem ConvNets solely on 32×32 CIFAR images collapses their final feature maps to 1×1. To test spatial feature maps properly while keeping compute reasonable, Track V-A spans three image resolutions using standard ImageNet subsets alongside CIFAR (Table 3). Classification tasks (V-A1 through V-A4) are scored by **Top-1 Accuracy (%)**, **Top-5 Accuracy (%)**, and **Validation Cross-Entropy Loss ($\mathcal{L}_{\text{val}}$)**:

V-A1. Residual CNN (`ResNet-50` [28], CIFAR 3×3 stem, $\approx$ 23.7M params): Standard residual network with `BatchNorm2d` and `ReLU` trained on **CIFAR-100** (32×32, 50,000 train / 10,000 val images, 100 classes) [4, 5, 29], using the `timm` `resnet50` backbone modified with a 3×3 CIFAR stem.

V-A2. Normalization-Free CNN (`NF-ResNet-50` [3], $\approx$ 25.6M params): Replaces batch normalization with Scaled Weight Standardization (`ScaledStdConv2d`) and trains at full 224×224 resolution on **ImageWoof** (10 fine-grained dog breed classes from ImageNet, 9,469 train / 3,925 val images) using the `timm` `nf_resnet50` model. With $\sim$ 9.5k training images (3,600 steps per 100 epochs at `batch_size=256` with `drop_last=True`), ImageWoof tests full 224×224 feature maps at roughly one-fifth the step count of CIFAR-100.

V-A3. Depthwise ConvNet (`ConvNeXt-V2-Tiny` [30], $\approx$ 28.6M params): Combines 7×7 depthwise convolutions, `LayerNorm`, and Global Response Normalization (GRN) under Mixup (0.8), CutMix (1.0), and Label Smoothing (0.1) on **Mini-ImageNet** (128×128, 100 ImageNet classes, 50,000 train / 10,000 val images) [9] via `timm` `convnextv2_tiny`. At 128×128 resolution, the final stage preserves a 4×4 feature grid for the 7×7 depthwise kernels with the same step count as CIFAR-100 (19,500 steps over 100 epochs).

V-A4. Vision Transformer (`ViT-Small` [31], Patch 8×8, $\approx$ 22.0M params): Multi-head self-attention backbone trained on **Mini-ImageNet** (128×128, 100 classes, 50,000 train / 10,000 val images) following the Mini-ImageNet ViT setup in Liang et al. [9]. Uses the `timm` `vit_small_patch16_224` architecture configured for 128×128 inputs and 8×8 patches (256 tokens per image) under RandAugment, Mixup (0.8), CutMix (1.0), and Label Smoothing (0.1).

V-A5. Flow-Matching Generative Model (`SiT-S/2` [26, 32], ≈ 33.0M params): Scalable Interpolant Transformer trained with Rectified Flow Matching velocity MSE loss ($\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,x_0,x_1}[\|v_\theta(x_t,t) - (x_1 - x_0)\|_2^2]$ and Model EMA 0.999) on **CIFAR-10** (32×32, $50,000$ train / $10,000$ val images). Evaluated by held-out **Velocity MSE Loss ($\mathcal{L}_{\text{FM}}$)** on the $10,000$ validation images and **Fréchet Inception Distance (FID@10k)** from $10,000$ ODE-sampled images (using a 50-step Heun/Euler solver on the 0.999 EMA weights; FID@50k in the Extended Setup).

Epoch Budget, Deterministic Shuffling, and Validation Schedule: Track V-A allows 50–150 epochs for CNNs (V-A1, V-A2) and 50–200 epochs for ConvNeXt, ViT, and SiT (V-A3–V-A5), with a **standard default of 100 epochs** ($19,500$ steps for V-A1/A3/A4/A5; $3,600$ steps for V-A2) at `batch_size = 256`. Training loaders shuffle sample indices each epoch using a deterministic generator seeded with `seed + epoch` and drop incomplete trailing batches, ensuring that all six optimizers see the same sequence of mini-batches (x_t, y_t) without class-order bias in `BatchNorm2d` or Mixup/CutMix. Every optimizer runs for the same epoch count E under linear warmup (5% of E for V-A1/A2/A5; 10% of E for V-A3/A4) followed by cosine decay to $\eta_{\min} = 0.05\eta$. Validation runs every 10% of total epochs and at the end of training over the full validation split, logging accuracy and loss alongside median step time (ms) and throughput (images/sec).

6.4 Track V-B: Downstream Fine-Tuning (Optional)

Track V-B is an optional suite for evaluating transfer learning from pretrained weights at small learning rates ($\eta \leq 10^{-4}$). At these step sizes, unrounded `bfloat16` additions can round to zero whenever $|\Delta\theta_{t,i}| = |\eta_t \delta_{t,i}| < \frac{1}{2}\text{ULP}(\theta_{t,i})$ [13], whereas poorly scaled updates can degrade pretrained features (Table 4):

V-B1. Object Detection Fine-Tuning (`RT-DETR-R18` / `YOLOv11-S` / `FCOS-ResNet50`):

- *Setup & Dataset:* Pretrained COCO or ImageNet backbone paired with a newly initialized detection head on **cppe-5** (5 classes, 480×480) or **Pascal VOC 2007** (20 classes, 480×480; or **MS-COCO 2017** 118k train / 5k val in Extended Setup).
- *Schedule & Learning Rates:* 15–35 epochs (**default 25 epochs**), `batch_size = 16` (or 32), 2-epoch linear warmup followed by cosine decay. Learning rates: Candidate $\eta \in \{1 \times 10^{-4}, 2 \times 10^{-4}\}$, AdamW $\eta = 1 \times 10^{-4}$, Lion $\eta = 3 \times 10^{-5}$.
- *Loss & Metrics:* Hungarian bipartite matching / multi-task bounding-box loss (Focal + L_1 + GIoU), scored every 5 epochs by **COCO mAP$_{50:95}$ (%)** (mean Average Precision averaged over IoU thresholds $0.50 : 0.05 : 0.95$), **mAP$_{50}$ (%)** (AP at IoU = 0.50), and **Validation Box Loss**.

V-B2. Fine-Grained Classification Fine-Tuning (`ViT-Base/16` & `ConvNeXt-Base`):

- *Setup & Dataset:* Pretrained ImageNet-1K backbone (`timm`) with a new linear classification head on **Oxford-IIIT Pet** (37 classes, $3,680$ train / $3,669$ test images at 224×224) [2].
- *Schedule & Learning Rates:* 10–30 epochs (**default 20 epochs**), `batch_size = 64`, 2-epoch linear warmup followed by cosine decay. Learning rates: Candidate $\eta = 1 \times 10^{-4}$, AdamW $\eta = 1 \times 10^{-4}$, Lion $\eta = 3 \times 10^{-5}$.
- *Metrics:* **Top-1 Accuracy (%)**, **Top-5 Accuracy (%)**, and **Validation Cross-Entropy Loss** measured every 2 epochs across all $3,669$ test images.

Table 3: Track V-A (Core): Vision From-Scratch Evaluation Matrix (`pretrained=False`, `bfloat16`)

Code	Model Backbone	Architecture	Dataset (Val)	Batch	Epochs	Warmup & Schedule	Learning Rate	Primary Metrics
V-A1	`ResNet-50` [28] (CIFAR 3×3 stem, ~ 23.7M)	Residual CNN (`BatchNorm2d` + `ReLU`)	CIFAR-100 (32^2) (50k train / 10k val)	256	50–150 Ep **100 Ep** (19.5k)	5% (5 Ep) Warmup + Cosine to 0.05η	Cand: 8e-4, 3e-4 AdamW: 1e-3 Lion: 1e-4	Top-1/Top-5 Acc, Val CE Loss, $R_{t,l}$
V-A2	`NF-ResNet-50` (Norm-Free [3], ~ 25.6M)	Norm-Free CNN (Scaled Weight Std, No BN)	ImageWoof (224^2) (9.5k train / 3,925 val)	256	50–150 Ep **100 Ep** (3.6k)	5% (5 Ep) Warmup + Cosine to 0.05η	Cand: 8e-4, 3e-4 AdamW: 1e-3 Lion: 1e-4	Top-1/Top-5 Acc, Val CE Loss, $R_{t,l}$
V-A3	`ConvNeXt-V2-T` (Tiny [30], ~ 28.6M)	Depthwise ConvNet (DW 7×7 + LN + GRN)	Mini-ImageNet (128^2) (50k train / 10k val)	256	50–200 Ep **100 Ep** (19.5k)	10% (10 Ep) Warmup + Cosine to 0.05η (Mix/Cut)	Cand: 8e-4, 3e-4 AdamW: 8e-4 Lion: 1e-4	Top-1/Top-5 Acc, Val CE Loss, $R_{t,l}$
V-A4	`ViT-Small` [9, 31] (Patch 8×8, ~ 22.0M)	Vision Transformer (Multi-Head Self-Attn)	Mini-ImageNet (128^2) (50k train / 10k val)	256	50–200 Ep **100 Ep** (19.5k)	10% (10 Ep) Warmup + Cosine to 0.05η (Rand/Mix/ Cut)	Cand: 8e-4, 3e-4 AdamW: 8e-4 Lion: 1e-4	Top-1/Top-5 Acc, Val CE Loss, $R_{t,l}$
V-A5	`SiT-S/2` [26] (Interpolant ViT, ~ 33.0M)	Generative Vision (Rectified Flow [32])	CIFAR-10 (32^2) (50k train / 10k val)	256	50–200 Ep **100 Ep** (19.5k)	5% (5 Ep) Warmup + Cosine to 0.05η (EMA 0.999)	Cand: 5e-4, 3e-4 AdamW: 5e-4 Lion: 1e-4	Held-Out Velocity MSE & FID@10k

Table 4: Track V-B (Optional): Vision Downstream Fine-Tuning Evaluation Matrix

Code	Pretrained Backbone	Downstream Task	Dataset & Val Split	Batch	Epochs (Range / Default)	Warmup & Schedule	Learning Rate (η)	Primary Metrics
V-B1	`RT-DETR-R18` / `YOLOv11-S` / `FCOS-R50`	End-to-End Object Detection (Hungarian / Focal + L_1 + GIoU)	cppe-5 or VOC 2007 (480^2; or COCO val2017 5k)	16	15–35 Ep **25 Ep** (Eval / 5 Ep)	2 Ep Linear Warmup + Cosine Decay	Cand: 1e-4, 2e-4 AdamW: 1e-4 Lion: 3e-5	COCO $\text{mAP}_{50:95}$, mAP_{50}, Val Loss
V-B2	`ViT-Base/16` [31] & `ConvNeXt-B` [30]	Fine-Grained High-Res Transfer Classification	Oxford-IIIT Pet (224^2) (3,680 train / 3,669 test)	64	10–30 Ep **20 Ep** (Eval / 2 Ep)	2 Ep Linear Warmup + Cosine Decay	Cand: 1e-4 AdamW: 1e-4 Lion: 3e-5	Top-1 & Top-5 Acc, Val CE Loss

6.5 Acceptance Criteria for Vision (V-Eval)

Consistent with Stage IV, V-Eval evaluates candidates under two memory classes against the two-buffer baselines (`AdamW_Native_BF16` and `AdamW_FP32_Master`) and single-buffer baselines (`Lion_Native_BF16` and `Lion_FP32_Master`), where $\Delta X = X_{\text{cand}} - X_{\text{ref}}$:

Class I. Dual-Buffer / Full-State Optimizers (`Candidate_Class_I`): Passes Class I if update ratios and loss curves stay within $F_1 \wedge F_2$ bounds ($R_{t,l} \in [10^{-7}, 0.10]$ without unrecovered spikes), isolated optimizer VRAM does not exceed `AdamW_Native_BF16` ($\Delta\text{VRAM}_{\text{opt}} \leq 0.0$ GB at 0.01 GB resolution), median isolated `opt.step()` time stays within 1.15× of `AdamW_Native_BF16`, and validation quality matches or exceeds `AdamW_Native_BF16` on the target task:

$$\begin{gathered}\Delta\text{Acc}_{\text{Top-1}} \geq 0.0\%, \quad \Delta\text{Acc}_{\text{Top-5}} \geq -0.2\%, \quad \Delta\text{mAP}_{50:95} \geq 0.0\%, \\ \Delta\mathcal{L}_{\text{FM}} \leq 0.0, \quad \Delta\text{FID} \leq 0.0.\end{gathered} \tag{12}$$

Class II. Single-Buffer / Compressed-State Optimizers (`Candidate_Class_II`): Passes Class II if it satisfies $F_1 \wedge F_2$ stability, cuts isolated optimizer memory by at least 35.0% compared to `AdamW_Native_BF16` and at least 70.0% compared to `AdamW_FP32_Master` while staying within +10.0% of single-buffer `Lion_Native_BF16`:

$$\begin{gathered}\Delta\text{VRAM}_{\text{opt}}^{(\text{vs. AdamW-BF16})} \leq -35.0\%, \quad \Delta\text{VRAM}_{\text{opt}}^{(\text{vs. AdamW-FP32})} \leq -70.0\%, \\ \Delta\text{VRAM}_{\text{opt}}^{(\text{vs. Lion-BF16})} \leq +10.0\%,\end{gathered} \tag{13}$$

keeps median isolated `opt.step()` time within 1.15× of `AdamW_Native_BF16` (and faster than `AdamW_FP32_Master`), remains within $\Delta\mathcal{L}_{\text{val}} \leq 0.25$ nats (or −1.5% Top-1 accuracy) of two-buffer `AdamW_Native_BF16`, and matches or outperforms single-buffer `Lion_Native_BF16` ($\Delta\text{Acc}_{\text{Top-1}}^{(\text{vs. Lion-BF16})} \geq 0.0\%$ or $\Delta\mathcal{L}_{\text{val}}^{(\text{vs. Lion-BF16})} \leq 0.05$ nats).

7 Version Regression Testing (D-Eval)

Whenever an update rule or compute kernel changes from version v_{k-1} to v_k, **D-Eval** ensures the revision introduces no mathematical drift, broken checkpoints, or latency regressions.

7.1 Regression Verification Suite

Before a new release is merged, candidate version v_k must pass the four verification gates summarized in Table 5.

7.2 Performance Delta Comparison

When running Check 4, the new version v_k is compared directly against v_{k-1}:

$$\Delta\mathcal{L}_{\text{EMA}} \leq 0, \qquad \Delta\text{VRAM}_{\text{opt}} \leq 0, \qquad \Delta T_{\text{kernel}} \leq 0, \tag{14}$$

where $\Delta X = X(v_k) - X(v_{k-1})$. Any regression ($\Delta > 0$) must be explained and recorded in the project documentation before applying the change.

Table 5: The D-Eval Four-Check Regression Checklist

Check	Evaluation Target	Acceptance Criterion	Verification Tool
Check 1	Algebraic Safety	All Stage I SMT queries return `UNSAT`; function approximation and stochastic rounding error bounds hold.	Z3 [1] / SymPy [10]
Check 2	Synthetic & Backend Parity	2D convergence on $\kappa \geq 1000$ and cross-backend/sharded numerical parity checks pass within tolerance.	2D Sim & Backend Tests
Check 3	Checkpoint Compatibility	State dictionaries saved under v_{k-1} load cleanly into v_k with identical tensor dtypes and step resumption.	State-Dict Check
Check 4	Loss, Memory, and Latency	Meets non-regression bounds: $\Delta\mathcal{L} \leq 0$, $\Delta\text{VRAM}_{\text{opt}} \leq 0$, $\Delta T_{\text{kernel}} \leq 0$ (optional when the update math is unchanged).	Stage IV (Optional)

References

[1] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Budapest, Hungary, 2008, pp. 337–340.

[2] X. Chen *et al.*, "Symbolic discovery of optimization algorithms," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 49205–49233.

[3] A. Brock, S. De, S. L. Smith, and K. Simonyan, "High-performance large-scale image recognition without normalization," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 1059–1071.

[4] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA, 2019.

[5] J. Zhuang, T. Tang, Y. Ding, S. Tatikonda, N. Dvornek, X. Papademetris, and J. S. Duncan, "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 18795–18806.

[6] S. Taniguchi, K. Harada, G. Minegishi, Y. Oshima, S. C. Jeong, G. Nagahara, T. Iiyama, M. Suzuki, Y. Iwasawa, and Y. Matsuo, "ADOPT: Modified Adam can converge with any β_2 with the optimal rate," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, Vancouver, BC, Canada, 2024, pp. 72438–72474.

[7] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, Stockholm, Sweden, 2018, pp. 1842–1850.

[8] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian, "GaLore: Memory-efficient LLM training by gradient low-rank projection," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, vol. 235, Vienna, Austria, 2024, pp. 61121–61143.

[9] K. Liang, L. Chen, B. Liu, and Q. Liu, "Cautious optimizers: Improving training with one line of code," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[10] A. Meurer *et al.*, "SymPy: Symbolic computing in Python," *PeerJ Comput. Sci.*, vol. 3, p. e103, Jan. 2017, doi: 10.7717/peerj-cs.103.

[11] A. Molina, P. Schramowski, and K. Kersting, "Padé activation units: End-to-end learning of flexible activation functions in deep networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[12] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, vol. 37, Lille, France, 2015, pp. 1737–1746.

[13] P. Zamirai, J. Zhang, C. R. Aberger, and C. De Sa, "Revisiting BFloat16 training," *arXiv preprint arXiv:2010.06192v2*, Mar. 2021.

[14] M. Jamil and X.-S. Yang, "A literature survey of benchmark functions for global optimization problems," *Int. J. Math. Model. Numer. Optim.*, vol. 4, no. 2, pp. 150–194, 2013, doi: 10.1504/IJMMNO.2013.055204.

[15] H. H. Rosenbrock, "An automatic method for finding the greatest or least value of a function," *Comput. J.*, vol. 3, no. 3, pp. 175–184, Oct. 1960, doi: 10.1093/comjnl/3.3.175.

[16] L. A. Rastrigin, *Systems of Extremal Control*. Moscow, Russia: Nauka, 1974.

[17] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, New Orleans, LA, USA, 2022, pp. 16344–16359.

[18] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. 3rd ACM SIGPLAN Int. Workshop Mach. Learn. Program. Lang. (MAPL)*, Phoenix, AZ, USA, Jun. 2019, pp. 10–19, doi: 10.1145/3315508.3329973.

[19] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, Feb. 2024, doi: 10.1016/j.neucom.2023.127063.

[20] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 12360–12371.

[21] G. Penedo, H. Kydlíček, L. Ben Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf, "The FineWeb datasets: Decanting the web for the finest text data at scale," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, Vancouver, BC, Canada, 2024, pp. 30811–30849.

[22] I. Magnusson *et al.*, "Paloma: A benchmark for evaluating language model fit," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, Vancouver, BC, Canada, 2024, pp. 64333–64376.

[23] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, "Gated linear attention transformers with hardware-efficient training," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, vol. 235, Vienna, Austria, 2024, pp. 56501–56523.

[24] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[25] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 8024–8035.

[26] N. Ma, M. Goldstein, M. S. Albergo, N. M. Boffi, E. Vanden-Eijnden, and S. Xie, "SiT: Exploring flow and diffusion-based generative models with scalable interpolant transformers," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Milan, Italy, 2024, pp. 23–40.

[27] J. Zhu, X. Chen, K. He, Y. LeCun, and Z. Liu, "Transformers without normalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Nashville, TN, USA, Jun. 2025, pp. 14901–14911, doi: 10.1109/CVPR52734.2025.01388.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[29] M. R. Zhang, J. Lucas, G. E. Hinton, and J. Ba, "Lookahead optimizer: k steps forward, 1 step back," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 9593–9604.

[30] S. Woo *et al.*, "ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Vancouver, BC, Canada, 2023, pp. 16133–16142.

[31] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[32] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, 2023.

Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval v1.1.1)

Technical Specification — Lumina Moon (`cloverx-id`)